

UNIT:2

Relational Query Languages: Relational Operations. Relational Algebra, Selection and projection set operations, renaming, Joins, Division, Examples of Algebra overviews, Relational calculus, Tuple relational Calculus, Domain relational calculus, Extended relation algebra operation

RELATIONAL MODEL CONCEPTS:

In 1970s the relational model was first introduced by Ted Codd in IBM Research. The model uses the concepts of mathematical relation.

The relational model represents the database as a collection of relations where each relation is nothing but a table with rows and columns.

Relation: A set of records in the form of a table is called a relation.

A relation contains 2 things:

1) relation schema 2) relation instance.

Relation Schema: It contains the basic information of a table. This information includes the name of the table, names of the attributes, datatypes, ranges, primary key etc.

Ex: student (studid: string primary key, name: string, age: integer)

Relation Instance:

At a particular instance of time, all the available records or tuples of a table is called relation instance.

Domain:

A domain D is a set of atomic values. For each attribute there is a set of permitted values called the domain of that attribute.

Relation Cardinality:

The no. of records or rows available in a relation is called relation cardinality.

Relation Degree:

The no. of attributes in a relation is called Relation Degree.

Tuple:

Each row or record of a table called Tuple.

Relational Database Schema: It a collection of relation schemas.

Relational Model Constraints:

These are the restrictions or sets of rules imposed on the database contents. It validates the quality of the database. Mainly Constraints on the relational database are of 4 types

- 1) Domain constraints
- 2) Key constraints
- 3) Entity Integrity constraints
- 4) Referential integrity constraints

1. Domain Constraints

Every domain must contain atomic values(smallest indivisible units) which means composite and multi-valued attributes are not allowed.

Example:

EMP ID	Name	Phone
01	Alok kumar shah	1234567890 2344566780

In the above relation, Name is a composite attribute and Phone is a multi-values attribute, so it is violating domain constraint.

Correct way is:

EMP ID	First Name	Mid Name	Last Name	Phone1	Phone2
01	Alok	kumar	shah	1234567890	2344566780

2. Key Constraints or Uniqueness Constraints

- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- A relation can have multiple keys or superkeys, out of which we choose one of the keys as the primary key.
- Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint.

Example:

<u>EID</u>	Name	Phone	DL	PAN
01	Bikash	6000000009	Kjkl4564	Qqqq1111
02	Paul	9000090009	Qwe12656	Wwww2222
04	Tuhin	9234567892	ASD87612	Eeee4444

In the above table, EID is the primary key, and the first and the last tuple have the same value in EID ie 01, so it is violating the key constraint.

$T1[\text{SuperKey}] \neq T2[\text{Superkey}]$

Where T1 and T2 are Tuples.

Super Key:

Every relation must have atleast one default super key.

A super key is a set of one or more attributes that uniquely identifies each tuple in a relation.

So every relation has one default super key ie: set of all its attributes.

Example:

Consider a relation: Students

Attributes: Students(rollno, name, age, DL)

Key attribute is rollno

Possible super keys are:

- 1) (rollno, name, age, DL)
- 2) (rollno, age, DL)
- 3) (rollno, name, DL)
- 4) (rollno, DL)
- 5) (DL)
- 6) (rollno)

Candidate Keys:

Every key constraint of a relation can be called a candidate key.

Candidate key is an attribute that can uniquely identify a tuple in a relation.

Ex: rollno, adhaarno, panno, driving licence no, registration no, passport no, voter id, bank account no etc.

Primary Key:

In a relation one of the candidate keys is used as a primary key.

A primary key is used to uniquely identify each tuple in a relation and it cannot have NULL value.

Constraints for NULL value:

It is a constraints on attribute which specifies whether NULL values are permitted or not.

Entity Integrity Constraints:

=====

It states that no primary key value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having NULL values for the primary key implies that we cannot identify some tuples.

Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	6000000009
NULL	Sony	9234567892

Referential Integrity:

=====

The Referential integrity constraint is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.

This constraint is seen when a primary key attribute of a relation R1 is used for referring in another relation R2. Here that attribute in R2 is called as Foreign key.

Example:

relation : Products, primary key : PNO

PNO	PNAME	QTY
12	LUX	100
13	MAGGI	200
14	THUMSUP	150

Relation : Sales, primary key : EID, foreign key : ProdNO

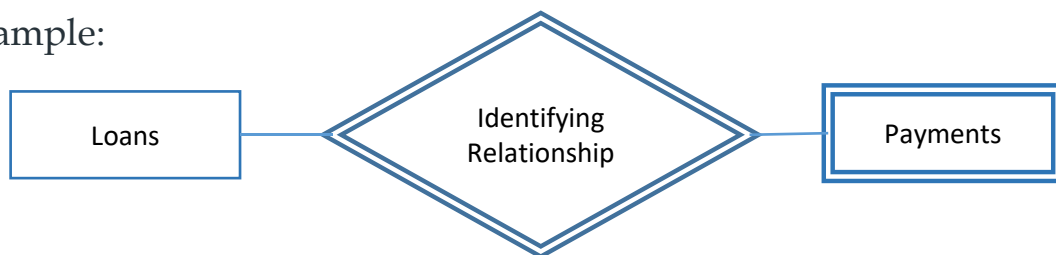
EID	Cust Code	Date	ProdNO	Qty
01	C1	1/1/25	12	10

EID	Cust Code	Date	ProdNO	Qty
02	C2	2/1/25	13	20
03	C1	2/1/25	13	10
04	C3	3/2/25	14	20
05	C2	4/3/25	12	40
06	C4	4/4/25	17	50

Partial Key:

The attribute of a weak entity set, that is associated with the primary key of a strong entity set to form the primary key of weak entity set, then it is known as partial key.

Example:



Relation: Loans, Primary Key : Loan No

Loan No	Loan Type	Amount	Customer ID

Relation: Loan Payments, Primary Key: Loan No + Pay SL No

Loan No	Pay SL No	Date	Amount

ER MODEL TO RELATIONAL MODEL MAPPING:

To convert ER Model to relational model we can follow the steps as:

- 1) Mapping of regular entities
- 2) Mapping of weak entity set
- 3) Mapping of 1:1 Binary relationship
- 4) Mapping of 1:N relationship
- 5) Mapping of Multivalued attributes
- 6) Mapping of M:N relationship
- 7) Mapping of N-Ary relationship

1) Mapping of regular entities:

=====

For each regular entity type in the ER diagram, create a corresponding table or relation. The table should include all the simple attributes of the entity type, and one of the key attributes should be designated as the primary key.

Example:

Consider a banking database system having entities: **branches**, **customers**, **employees**, **accounts** and **loans**.

Let us create tables for each entities:

Relation: branches

Bcode	Name	City	assets

Relation: Customers

Custid	Cname	address

Empid	Name	Dno	Street	City	Pin	doj	phone

Relation: employees

Relation: Accounts

Acno	Ac type	balance

Relation: loans

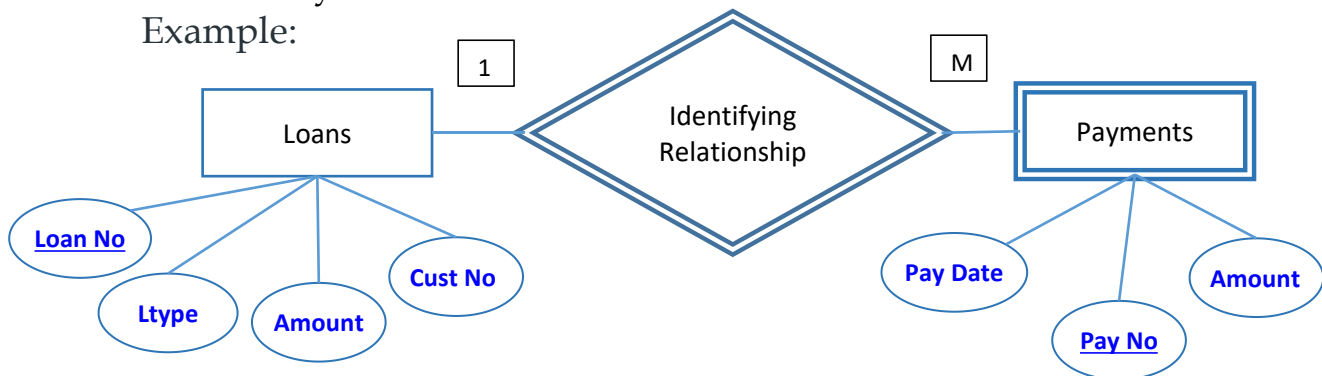
Loan no	Ltype	amount

2) Mapping of weak entity set:

=====

We need to include the primary key of strong entity as foreign key in weak entity.

Example:



Relation: Loans

<u>Loan No (PK)</u>	Ltype	Amount	Cust No

Relation: Payments

<u>Pay No (PK)</u>	Pay Date	Loan No (FK)	Amount

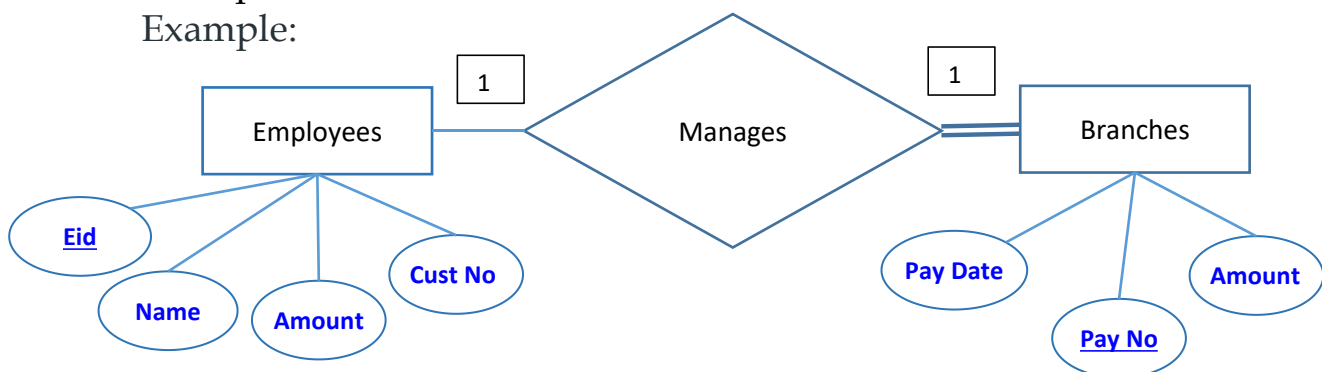
3) Mapping of 1:1 Binary relationship:

=====

In 1:1 mapping, if an entity having partial participation with another entity then the entity of total participation needs to include foreign key.

Example:

Example:



Relation: Employees

<u>Eid</u>	Name	Address	Phone

1	Abc		
2	QWE		
3	ZXC		

Relation: Branches

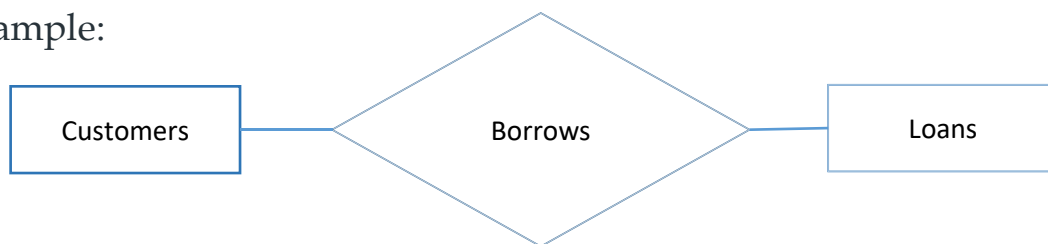
Bcode	Bname	Location	ManagerID
B1	CSE		2

4) Mapping of 1:N relationship:

=====

In 1:N relationship, the primary key of 1:relationship to be used as foreign key of N:relationship.

Example:



Here the primary key CustID is used as foreign key CID in loans relation.

Relation: Customer, Primary Key : CustID

CustID	Name	Address

Relation: Loans, Primary Key: LoanNo

LoanNo	Ltype	Amount	CID

5) Mapping of Multi valued attributes:

=====

Either create a separate relation for the multi valued attribute along with primary key or divide the multi valued attribute into different partitions.

Example1:

Relation: Employees

SLNo	EmpID	Phone
------	-------	-------

1	E1	111
2	E1	222
3	E1	333

Example2:

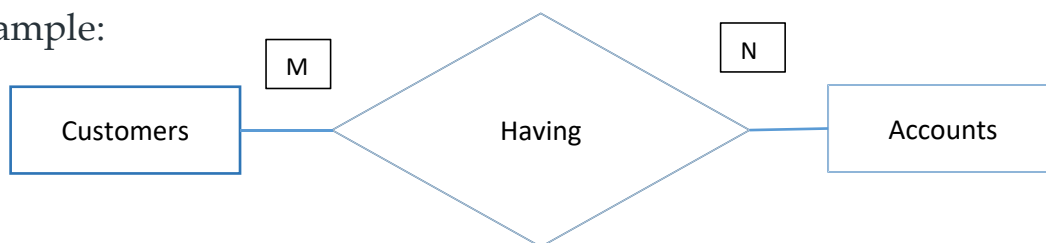
EmpID	Phone1	Phone2	Phone3
E1	111	222	333

6) Mapping M:N Relationship:

=====

Create a separate relation which will contain only primary keys of both entities as attributes

Example:



Relation 1: Customers

CustID	Name	Address

Relation 2: Accounts

AC No	AC Type	Balance

Relation 3: Cust_AC

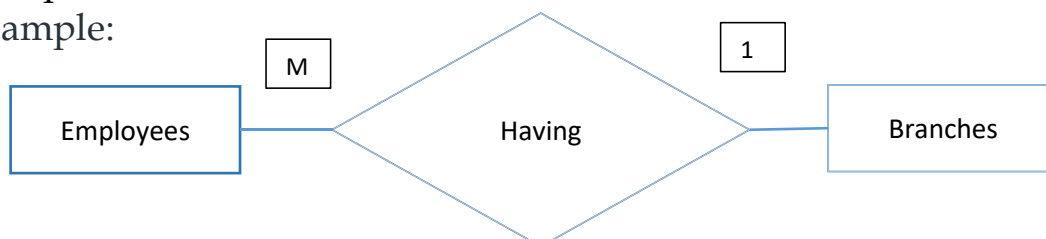
SLNo	CustID	ACNo

7) Mapping N-Ary Relationship:

Create a separate relation that includes all the primary keys of entity sets.

Example:

Example:





SLNo	EmpID	BranchID	ProjectID

=====

CASE STUDY: A SCHEMA DIAGRAM OF A COMPANY

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 3.5
Schema diagram for the
COMPANY relational
database schema.

Figure 3.6
One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1968-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
-------	---------	---------	----------------

DEPT_LOCATIONS

Dnumber	Dlocation
---------	-----------

PROJECT

Pname	Pnumber	Plocation	Dnum
-------	---------	-----------	------

WORKS_ON

Essn	Pno	Hours
------	-----	-------

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
------	----------------	-----	-------	--------------

Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

****These shall be followed in Lab for SQL Query Implementation.**

=====

CHAPTER : QUERY Languages:

A language which allows users to send a request or query to the database for retrieving some information.

Query language types:

- 1) Procedural
- 2) Non-Procedural

Procedural Language:

Here we have to specify the way of retrieving information from the database. Example: **Relational Algebra.**

Non-Procedural Language:

Here we can retrieve the information without specifying the procedure to retrieve. Example: **Relational Calculus.**

The query language explains the process of querying based on which the different programming languages operates such as: MY SQL, PL SQL, ISQL etc.

Relational Algebra:

A data model must include a set of operations to manipulate the database. **The basic set of operations for a relational model is called relational algebra.**

Relational Algebra is a formal language used to query and manipulate relational databases, consisting of a set of operations like selection, projection, union, and join. It provides a mathematical framework for querying databases, ensuring efficient data retrieval and manipulation. Relational algebra serves as the mathematical foundation for query SQL.

A sequence of relational algebra operations forms a relational algebra expression.

The fundamental operations in the relational algebra are:

- 1) Unary operations**
- 2) Binary operations**

Unary operations:

The operations are called unary when they operate on one relation.

They are:

- i) Select operation**
- ii) Project operation**
- iii) Rename operation**

Select Operation:

Here we use the lower case greek letter sigma (σ) to denote selection. The selection operation is used to select a subset of tuples from a relation based on a condition.

Syntax:

$\sigma_{\text{selection_condition}}(\text{relation_name})$

The selection_condition formula may use connectors such as or, and, and not. Also, these terms may make use of relational operators such as $=$, $\neq$, $\geq$, $<$, $>$, $\leq$.

For Example: Consider a Table Customers having attributes Custcode, Name and Location.

Relation: customers

Custcode	Name	location
01	sameer	bbsr
02	kiran	gnpr
03	tapan	kolkata
04	sushil	bbsr
05	kaustav	rgda
06	soumya	bbsr

Ex1: Find the customer details who are living in “BBSR” from the relation “customers”

Query:

$\sigma_{\text{location} = \text{“bbsr”}}$ (customers)

Ex2: to retrieve all the details of a relation:

Query:

σ (customers)

The selection condition may use the relational operators I.e: $>$, $<$, $\geq$, $\leq$, $\neq$, $=$

Also we can join multiple conditions using logical operators:

and operator is AND

Or operator is OR

Not operator is NOT

For Example2: consider a students relation having attributes: Studid, Name, City, Phone, Branch, percentage

Relation: students

Studid	Name	City	Phone	Branch	percentage
--------	------	------	-------	--------	------------

1	soumya	bbsr	111	ece	85
2	kiran	gnpr	222	ece	95
3	sourabh	gnpr	333	mech	85
4	abhi	bbsr	444	ece	65
5	srikant	bbsr	555	mech	80
6	kanthi	rgda	666	ece	82
7	sunita	rgda	777	cse	93
8	tarun	bbsr	888	mech	70
9	kanha	bbsr	999	ece	75
10	santi	gnpr	000	cse	95

Now consider the following queries on it:

Query1: Find the details from relation students where branch is ece.

σ branch = "ece" (students)

Query2: Find the details where percentage > 80

σ percentage > 80 (students)

Query3:

Find the details where branch is ece and belongs to bbsr

σ branch="ece" AND city="bbsr" (students)

Query4:

Find the details where branch is mech or percentage >= 85

σ branch="mech" OR percentage >= 85 (students)

Query5: Find the details who are not belongs to bbsr.

σ NOT (city="bbsr") (students)

(OR)

σ city != bbsr (students)

Some sample queries for practice:

=====

Q1) Find the customer details who lives at bbsr and having balance in account > 1000 rupees from relation "customers"

σ city = "bbsr" AND balance > 1000 (customers)

Q2) Find the employees belongs to maintenance dept having salary more than 20000 from relation "EMP"

σ dept = "maintenance" AND salary > 20000 (EMP)

It can also be written as:

σ dept = "maintenance" (σ salary > 20000 (EMP))

Its syntax is:

σ condition1 (σ condition2 (R))

OR

σ condition2 (σ condition1 (R))

OR

σ condition1 AND condition2 ®

Projection Operation:

=====

The projection operation selects certain columns from the table and discards the other columns.

The symbol used is upper case greek letter pi i.e: Π

Syntax :

Π <attributes list > (relation name)

The projection is nothing but a vertical partition of the relation.

Sample Queries:

=====

Q1) display only the studentID and percentage from the relation "students"

Query:

Π studentID, percenatge (students)

Q2) Display name and city from relation "students"

Query:

Π name, city (students)

Composition of selection and projection:

=====

We can combine both selection and projections in queries.

Sample queries:

Q1) Find the studentID and name where branch is ECE from relation students.

Query:

Π studentID, Name (σ branch="ece" (students))

Q2) Find the rollno and name from student relation where branch is IT and age ≥ 21

Π rollno, name (σ branch="IT" AND age ≥ 21 (students))

Π rollno, name (σ branch="IT" (σ age ≥ 21 (students)))

Q3) Find the students name where city is bbsr and branch is ECE.

Π name (σ branch="ECE" AND city="BBSR" (students))

Q4: Find name, phone number where branch is ECE and percentage ≥ 85 from relation: students.

Π name, phoneno (σ branch="ECE" AND percent ≥ 85 (students))

RENAME OPERATION:

It is used to assigned a new name to a table or relation. It is denoted by using a lowe case greek letter rho. (ρ).

Syntax:

$\rho\langle\text{newName}\rangle \text{ (RelationExpression)}$

Alternative syntax:

$\text{newName} \leftarrow (\text{RelationExpression})$

Ex:

$\rho \text{ employee } (\sigma (\text{emp}))$

Ex:

$\rho \text{Sales_emp } (\sigma \text{ dept} = \text{"sales"} (\text{emp}))$

Rename is also used for renaming attributes of a relation.

Example: We can rename an attribute **B** in relation **R** to **D**

$\rho(D/B) R$

will rename the attribute 'B' of the relation by 'D'.

Sample Queries:-

Q1) Create an alternate table for all the tuples of relation Emp.

Ans:

$\rho \text{ staff } (\sigma (\text{emp}))$

(OR) $\text{staff} \leftarrow (\sigma (\text{emp}))$

Q2) Assign a new table for students rollno, name where branch is EEE.

$\rho \text{ ECE_list } \left(\Pi_{\text{rollno}, \text{name}} (\sigma_{\text{branch} = \text{"EEE"}} (\text{student})) \right)$

(OR) $\text{ECE_list} \leftarrow \left(\Pi_{\text{rollno}, \text{name}} (\sigma_{\text{branch} = \text{"EEE"}} (\text{student})) \right)$

Q3) Assign new table for the students name, address who are belongs to BBSR and branch is ECE.

$\rho_{stud_ece_bbsr}(\pi_{name, address}(\sigma_{City="BBSR" \text{ AND } Branch="ECE"}(Students)))$

Q4) Create a relation called stud_data that contains studid, name where branch is ECE OR percentage ≥ 85

$\checkmark \text{ stud_data} \leftarrow (\pi_{studid, name}(\sigma_{branch="ECE" \text{ OR } Percent \geq 85}(Students)))$

We can break the complex sequence of operations by specifying temporary names for intermediate results.

ex) Create a relation called stud_data that contains studid, name where branch is ECE OR percentage ≥ 85

Answer:
 $\text{temp} \leftarrow (\sigma_{branch="ECE" \text{ OR } Per \geq 85}(Students))$

$$\text{Stud_data} \leftarrow (\pi_{\text{studid}, \text{name}}(\text{temp}))$$

✓ we can even use rename operation for renaming the attributes of alternate relation created.

Example:

Create a relation called stud having attributes: rollno, loc. It should contain data studid, city from students relation whose branch is Mech and percent ≥ 80 .

$$\text{temp} \leftarrow (\sigma_{\text{branch}=\text{"Mech"} \text{ AND } \text{Per} \geq 80}(\text{students}))$$

$$\text{stud}(\text{rollno}, \text{loc}) \leftarrow (\pi_{\text{studid}, \text{city}}(\text{temp}))$$

Here studid is renamed as rollno
City is renamed as loc.

Binary Operations:

The operations are known as binary when they operate on a pair of relations.

They are:

- i) Union operations
- ii) Intersection operations
- iii) Set difference
- iv) Cartesian product
- v) Natural join
- vi) Division
- vii) Assignment

Before we apply the binary operations on two relations **here we need to check for** the compatibility known as **Tuple compatibility**.

Two relations $R(a_1, a_2, a_3, \dots, a_n)$ and $S(b_1, b_2, b_3, \dots, b_n)$ are union compatible, if they have same degree and if $\text{domain}(A_i) = \text{domain}(B_i)$

That means the no. of attributes and type of attributes must match in both the relations.

Union operations:

=====

It is an operation from set theory denoted by $\cup$. It is used to combine two relations ex: $R \cup S$.

The result of this operation contains all the tuples that are either in R or in S or in both. Here the duplicate tuples are eliminated.

$Q = R \cup S$ then, Q will contain all the tuples of R and S by excluding duplicacy.

Example:

Relation: Java(rollno, name, address)

Relation: Python(rollno, name, address)

Relation: Java(rollno, name, address)

rollno	name	address
--------	------	---------

j01	Soumya	bbsr
-----	--------	------

j02	Kiran	Gnpr
-----	-------	------

j03	Shivani	bbsr
-----	---------	------

Relation: Python(rollno, name, address)

rollno	name	address
--------	------	---------

p01	Tina	bbsr
-----	------	------

p02	Meena	bbsr
-----	-------	------

j03	Shivani	bbsr
-----	---------	------

p03	Kishan	Gnpr
-----	--------	------

Operation:

Query1: Java $\cup$ Python

Result (Union of Java and Python):

rollno name address

j01 Soumya bbsr

j02 Kiran Gnpr

j03 Shivani bbsr

p01 Tina bbsr

p02 Meena bbsr

p03 Kishan Gnpr

*** Here duplicate is eliminated.**

Example2:

Query2

$\Pi_{rollno}(Java) \cup \Pi_{rollno}(Python)$

rollno

j01

j02

j03

rollno

p01

p02

j03

p03

Result:

rollno

j01

j02

j03

p01

p02

rollno

p03

*** Duplicate j03 is eliminated.**

Example3:

Query3:

$\Pi_{\text{address(Java)}} \cup \Pi_{\text{address(Python)}}$

$\Pi_{\text{address(Java)}}$

address

bbsr

Gnpr

bbsr

$\Pi_{\text{address(Python)}}$

address

bbsr

bbsr

bbsr

Gnpr

Result: $\cup$

address

bbsr

Gnpr

Only two unique addresses remain.

Ex4:

Suppose we have two different relations that do not directly match:

Students: (studid, name, address, phone, age)

Results: (studid, name, percent, grade)

We can only apply union after projecting common attributes:

Query4:

$\pi_{studid,name}(\text{Students}) \cup \pi_{studid,name}(\text{Results})$

INTERSECTION OPERATION:

=====

It takes two relations as input and produces a resultant relation that contains common tuples from both tables. It is denoted by $\cap$.

Suppose $Q = R \cap S$, then Q will contains the common tuples of both R and S.

Example1:

Query1:

$\text{Java} \cap \text{Python}$

Display the common list of address in both the relations:

rollno	name	address
j03	Shivani	bbsr

Example2:

Query2:

$\pi_{\text{address}}(\text{Java}) \cap \pi_{\text{address}}(\text{Python})$

Result:

address

bbsr

Gnpr

Set difference operation (Minus)

=====

It is a binary operation which takes two relations as an input and
It finds the tuples that are available in one relation but not in
another relation.

Ex1:

Suppose $R = \{1,2,3,4,5\}$ and $S = \{3,5,9,11\}$

If $Q = R - S$ Then $Q = \{1,2,4\}$

If $Q = S - R$ Then $Q = \{9,11\}$

Ex:

Java – Python $\rightarrow$ Tuples in Java but not in Python

Python – Java $\rightarrow$ Tuples in Python but not in Java

Query1:

Java – Python

Result:

rollno	name	address
--------	------	---------

j01	Soumya	bbsr
-----	--------	------

j02	Kiran	Gnpr
-----	-------	------

Ex:

Python – Java

Result:

rollno	name	address
--------	------	---------

p01	Tina	bbsr
-----	------	------

p02	Meena	bbsr
-----	-------	------

p03	Kishan	Gnpr
-----	--------	------

Ex: Find the address from relation: java which is not exist in relation : python

Table: java

rollno	name	address
--------	------	---------

j01	Soumya	bbsr
-----	--------	------

j02	Kiran	Gnpr
-----	-------	------

j03	Shivani	bbsr
-----	---------	------

j04	Sam	rgda
-----	-----	------

Table : python

rollno	name	address
--------	------	---------

p01	Tina	bbsr
-----	------	------

p02	Meena	bbsr
-----	-------	------

rollno	name	address
j03	Shivani	bbsr
p03	Kishan	Gnpr

Query2:

$\pi_{\text{address}}(\text{Java}) - \pi_{\text{address}}(\text{Python})$

steps:

Step 1: Projection on address

From Java $\rightarrow \{ \text{bbsr, Gnpr, rgda} \}$

From Python $\rightarrow \{ \text{bbsr, Gnpr} \}$

Step2:

$\{ \text{bbsr, Gnpr, rgda} \} - \{ \text{bbsr, Gnpr} \}$

Step3:

$\{ \text{rgda} \}$

Result:

address
rgda

Problem:

Consider relation:

Students(studid, name, branch, address, age, percent, phone).
Find the student details where branch is not ECE.

Step 1: Extract ECE students

$\text{stud_ece} \leftarrow (\sigma_{\text{branch} = \text{"ECE"}} (\text{Students}))$

Step 2: Students NOT in ECE

$\text{NONECE} = \text{Students} - \text{stud_ece}$

Find only the IDs and names of students who are not in ECE.

Query:

$\text{stud_ece} \leftarrow (\sigma_{\text{branch} = \text{"ECE"}} (\text{Students}))$

NONECEdata ← **($\pi_{studid, name}(Students)$)** – **($\pi_{studid, name}(stud_ece)$)**

Cartesian Product (X)

=====

We already are aware of the fact that relations are nothing but a set of tuples, and here we will have 2 sets of tuples.

On applying CARTESIAN PRODUCT on two relations that is on two sets of tuples, it will take every tuple one by one from the left set(relation) and will pair it up with all the tuples in the right set(relation).

If we have two relations:

the **CROSS PRODUCT (X)** of two relation **R(R1, R2, R3, ..., Rp)** with degree p, and **S(S1, S2, S3, ..., Sn)** with degree n, is a relation **T(R1, R2, R3, ..., Rp, S1, S2, S3, ..., Sn)** with degree p + n attributes.

Notation:

T = R × S

where R and S are the relations,
the symbol '×' is used to denote the CROSS PRODUCT operator.

R(R1, R2, R3, ..., Rp) and S(S1, S2, S3, ..., Sn)

Schema of T: T(R1,R2,R3,...,Rp,S1,S2,...,Sn)

Degree of T: p+n(sum of attributes of both relations)

Number of tuples: | R | × | S |

Example

Let's say:

Relation R (Students):

R1 (sid) R2 (sname)

S1 Soumya

S2 Kiran

S3 Tina

Relation S (Courses)

S1 (cid) S2 (cname) S3 (credits)

C1 Java 3

S1 (cid) S2 (cname) S3 (credits)

C2 Python 4

C3 DBMS 3

C4 OS 4

Cartesian Product: $T = R \times S$

sid sname cid cname credits

S1 Soumya C1 Java 3

S1 Soumya C2 Python 4

S1 Soumya C3 DBMS 3

S1 Soumya C4 OS 4

S2 Kiran C1 Java 3

S2 Kiran C2 Python 4

S2 Kiran C3 DBMS 3

S2 Kiran C4 OS 4

S3 Tina C1 Java 3

S3 Tina C2 Python 4

S3 Tina C3 DBMS 3

S3 Tina C4 OS 4

5 attributes (sid, sname, cid, cname, credits)

12 tuples (3×4)

EXAMPLE2:

Step1: Suppose given two relations: customers and borrowers

customers

Cid name Address

101 a gnpr

102 b rgda

104 c bbsr

108 d gnpr

110 e berh

borrowers

loanno Custid**L01 102****L02 104****L03 108****Step 2: Cartesian Product (Customer × Borrower)**

The result will have attributes:

(Customer.Cid, Customer.name, Customer.Address, Borrower.loanno, Borrower.Custid)

Since **Customer** has 5 rows and **Borrower** has 3 rows, the Cartesian product has $5 \times 3 = 15$ rows.

This includes many tuples where $\text{Customer.Cid} \neq \text{Borrower.Custid}$.

Step 3: Selection (Join Condition)

We apply condition:

$\sigma_{\text{Customer.Cid}=\text{Borrower.Custid}} (\text{Customer} \times \text{Borrower})$

Results is:

Customer.Cid	Customer.name	Customer.Address	Borrower.Custid	Borrower.loanno
102	b	rgda	102	L01
104	c	bbsr	104	L02
108	d	gnpr	108	L03

Step 4: Projection (Getting only desired attributes)

If we want only Customer.name and Borrower.loanno, apply projection:

$\pi_{\text{Customer.name}, \text{Borrower.loanno}} (\sigma_{\text{Customer.Cid}=\text{Borrower.Custid}} (\text{Customer} \times \text{Borrower}))$

Final Result:

Customer.name Borrower.loanno

b L01

Customer.name Borrower.loanno

c L02

d L03

Example 2: Retrieve a list of names of each female employee's dependents

Given Relations:

EMP(eno, ename, age, sex)

Dependent(dname, eeno, age, relation)

Step 1: Select Female Employees

$\text{emp_name} \leftarrow \pi_{\text{ename}, \text{eno}} (\sigma_{\text{sex}='F'} (\text{EMP}))$

This gives only **female employees** with their names and employee numbers.

Step 2: Cartesian Product + Selection

We match female employees with their dependents:

$\text{Act_depend} \leftarrow \sigma_{\text{eno}=\text{eeno}} (\text{emp_name} \times \text{Dependent})$

This ensures that we only keep dependents related to those female employees.

Join Operation:

=====

The join operation is denoted by 

It is used to combine two relations by **matching key attributes**.

A join can be written as:

$R \bowtie \langle \text{join condition} \rangle S$

The join condition will be denoted as: $A_i \theta B_j$

Where A_i is the attribute of R.

B_j is the attribute of S

And θ is the comparison operator

Example1: $\text{Customers} \bowtie (\text{Customers.CustID} = \text{Loans.CustID}) \text{Loans}$

Example2:

$\text{Temp} \leftarrow (\text{student}) \bowtie_{\text{student.roll}=\text{Exam.roll}} (\text{Exam})$

Example3:

Relation1: EMP(eno, ename, salary, dept)

Relation2: Dependents(depno, dname, eno)

$\text{emp_name} \leftarrow \Pi_{\text{eno}, \text{ename}} (\text{EMP})$

$\text{Emp_name}(\text{eno}, \text{ename})$

$\text{Dependents}(\text{depno}, \text{dname}, \text{eno})$

$(\text{Emp.eno}, \text{Emp.ename}, \text{dependent.depno}, \text{dependent.dname}, \text{dependent.eno})$

$\text{Result} \leftarrow \Pi_{\text{dname}, \text{ename}} (\text{emp_name} \bowtie_{\text{eno}=\text{eno}} \text{Dependent})$

Example 3:

Given relations:

Emp(eno, ename, address, salary)

Dept(dno, dname, summary, mgrno)

Task: Find employees who are managers of departments.

Relational Algebra Expression:

1. Join operation:

$$X \leftarrow Dept \bowtie_{mgrno=eno} Emp$$

(Join Dept and Emp where the manager number = employee number)

2. Projection:

$$\Pi_{dname,ename}(X)$$

(Select only department name and employee name from the result)

We join the Dept table with the Emp table based on the condition that the department's mgrno (manager number) matches the employee's eno (employee number).

From the result of this join, we project (Π) only the department name (dname) and employee name (ename) to get the required output: employees who are managers of departments, along with the department names.

Types of Join

There are many types of Joins in SQL. Here are the frequently used SQL JOIN types:

1. Inner Join

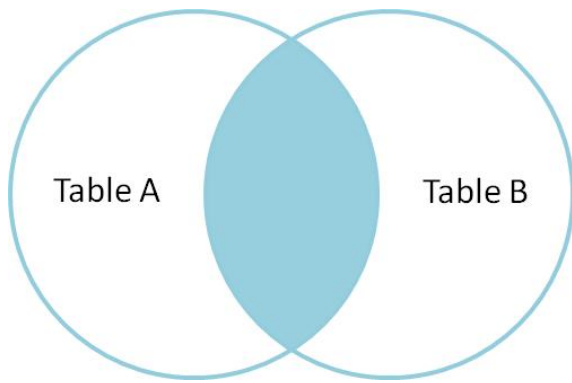
Inner Join is a join operation in DBMS that combines two or more tables based on related columns and returns only rows that have matching values among tables.

Inner join has three types.

1) Conditional join

2) Equi Join

3) Natural Join



(a) Conditional Join

Conditional join or Theta join is a type of inner join in which tables are combined based on the specified condition.

In conditional join, the join condition can include $<$, $>$, $<=$, $>=$, $\neq$ operators in addition to the '=' operator.

Example: Suppose two tables A and B

Table A

R	S
10	5
7	20

Table B

T	U
10	12
17	6

$A \bowtie_{S < T} B$

Output

R	S	T	U
10	5	10	12

R	S	T	U
10	5	17	6

(b) Equi Join

Equi Join is a type of inner join where the join condition uses the equality operator ('=') between columns.

Example: Suppose there are two tables Table A and Table C

$$A \bowtie_{A.ColumnB = C.ColB} C$$

(c) Natural Join

Natural join is a type of inner join in which we do not need any comparison operators.

In natural join, columns should have the same name and domain. There should be at least one common attribute between the two tables.

Ex:

$$A \bowtie B$$

It can also be written as :

$$A \bowtie B$$

2. Outer Join

Outer join is a type of join that retrieves matching as well as non-matching records from related tables. There are three types of outer join

- Left outer join
- Right outer join
- Full outer join

(a) Left Outer Join

This type of outer join retrieves all records from the left table and retrieves matching records from the right table.

Example: Suppose there are two tables Table A and Table B

Table A

Number	Square
2	4
3	9
4	16

Table B

Number	Cube
2	8
3	27
5	125

Query: $A \bowtie B$

Output

Number	Square	Cube
2	4	8
3	9	27
4	16	NULL

(b) Right Outer Join

This type of outer join retrieves all records from the right table and retrieves matching records from the left table.

Right Outer Join

Example: Suppose there are two tables Table A and Table B

$A \bowtie B$

Output:

Number	Square	Cube
2	4	8
3	9	27
5	NULL	125

(c) Full Outer Join

FULL JOIN creates the result set by combining the results of both LEFT JOIN and RIGHT JOIN. The result set will contain all the rows from both tables.

Example: Table A and Table B are the same as in the left outer join

$A \bowtie B$

Output:

Number	Square	Cube
2	4	8
3	9	27
4	16	NULL
5	NULL	125

DIVISION OPERATOR (%)

=====

Division in SQL is typically required when you want to find out entities that are interacting with **all entities** of a set of different types of entities.

The division operator is used when we have to evaluate queries that contain the keyword 'all'.

Here are a few common examples:

- Identifying people who have accounts in every bank within a particular city.
- Determining students who have enrolled in all necessary courses to qualify for graduation.

Example:

Emp(empid, ename, age, salary)

Find the **employeeid** of employee table who works on **all the projects** that "*Mahesh*" works on.

Given relation:

Emp(empid, name, age, salary)

Relations

- Emp(empid, name, age, salary)
- Projects(pno, pname, location)
- WorksOn(empid, pno, hours)

This models a many-to-many relationship between employees and projects.

Table: EMP

empid name age salary

101	Mahesh	21	1000
102	Kanti	31	2000
103	Kiran	41	3000
104	Soma	51	4000
105	Naina	61	5000

Table: Projects

pno pname location

P1	SS	gnpr
P2	PP	gnpr
P3	QQ	rgda
P4	RR	bbsr

Table: WorksOn

empid pno hours

101 P1 11

101 P3 2

101 P4 10

102	P1	5
-----	----	---

102	P4	10
-----	----	----

103 P1 10

103 P3 5

103 P4 10

104 P2 5

105 P4 15

QUERY:

Step 1:

$$mah \leftarrow \sigma_{fname="Mahesh"}(Emp)$$

- This selects tuples from **Emp** (Employee table) where the first name is *Mahesh*.
- Result: mah contains details of employee(s) named Mahesh.

O/P:

The relation mah contains Mahesh's details:

empid	name	age	salary
101	Mahesh	21	1000

Step 2:

$$mah_pno \leftarrow \pi_{pno} \left((works_on \bowtie_{empid=empid} mah) \right)$$

- Join **works_on** with **mah** (on empid).
- Project **pno** (project numbers).
- Result: **mah_pno** contains all projects on which Mahesh works.

O/P:

The projects that Mahesh (empid = 101) works on are:

pno

P1

P3

P4

Step 3:

$$Eno_pno \leftarrow \pi_{empid,pno}(works_on)$$

- Projects only employee IDs and project numbers from **works_on**.
- Result: **Eno_pno** is a relation showing which employee works on which project.

O/P:

The full employee-project assignments are:

empid	pno
101	P1
101	P3
101	P4
102	P1
102	P4
103	P1
103	P3
103	P4
104	P2
105	P4

Step 4:

$$Enos(empid) \leftarrow Eno_pno \div mah_pno$$

- Division operation: Find all **empid** in **Eno_pno** that are associated with **all the projects Mahesh works on**.
- Result: **Enos** contains IDs of employees who work on **all projects that Mahesh works on**.



O/P:

- **mah_pno = {P1, P3, P4}**
- We now find employees in **Eno_pno** who are assigned to **all three projects {P1, P3, P4}**.

Checking employee by employee:

- **101 (Mahesh) → Works on {P1, P3, P4} ✓**
- **102 → Works on {P1, P4}, but missing P3 ✗**
- **103 → Works on {P1, P3, P4} ✓**
- **104 → Works on {P2} only ✗**
- **105 → Works on {P4} only ✗**

Employees who work on **all projects that Mahesh works on**:

empid
101
103

Assignment Operation ($\leftarrow$)

- **Definition:**

It is used to assign the result of a **relation expression** to a **new variable**.

- **Example:**

$$Temp \leftarrow \pi_{empid, name}(Emp)$$

- Here, we project only **empid** and **name** from the relation **Emp**.
- The result is assigned to a new relation (temporary variable) called **Temp**.

—: Relational Calculus:—

Relational Calculus is a non-procedural language. It specifies what is to be retrieved rather than how to be retrieved.

A calculus expression does not contain how a query to be evaluated.

The Relational Calculus is of 2 types:

1. Tuple Relational Calculus (TRC)
(based on Tuple variables)
2. Domain Relational Calculus (DRC)
(based on domain variables)

Tuple Relational Calculus (TRC) in DBMS

Definition:

TRC is based on specifying a number of tuple variables.

A tuple variable may take any individual tuple from a relation as its value.

Syntax: The basic syntax of TRC is as follows:

$\{ t \mid \text{cond}(t) \}$

where: t is a tuple variable that represents a set of all tuples, $\text{cond}(t)$ is a conditional expression involving t .

Result: The set of all tuples t that satisfy $\text{cond}(t)$

Example (Q1)

Query:

Find all the employees whose salary is above 50000.

TRC Expression:

$\{ t \mid \text{Emp}(t) \wedge t.\text{sal} > 50000 \}$

Here:

t is a tuple variable representing tuples from the relation Emp.

Condition: $t.\text{sal} > 50000$.

Meaning: The TRC variable t represents a tuple such that:

$t \in \text{Emp}$ (belongs to Emp relation), and $t.\text{sal} > 50000$.

So the result = all employees with salary > 50000 .

Query 2

Retrieve first name and last name of employees whose salary is above ₹50,000.

TRC Expression:

$\{ \langle t.\text{fname}, t.\text{lname} \rangle \mid \text{emp}(t) \wedge t.\text{sal} > 50000 \}$

✓ Meaning: Select `fname` and `lname` from `Emp` where `sal > 50000`.

Query 3

Retrieve the date of birth and address of the employee whose name is "Mahesh".

TRC Expression:

$$\{\langle t.dob, t.address \rangle \mid emp(t) \wedge t.fname = "Mahesh"\}$$

✓ Meaning: From `Emp`, pick `dob` and `address` of employee(s) whose first name is "Mahesh".

Query 4

Find the names of male students who belong to the CSE branch and are older than 20.

TRC Expression:

$$\{\langle t.name \rangle \mid student(t) \wedge t.gender = "M" \wedge t.branch = "CSE" \wedge t.age > 20\}$$

✓ Meaning: From `Student`, return `name` of students who are male, in CSE branch, and older than 20.

Query 5

Find the name and phone number of customers who belong to "Delhi".

TRC Expression:

$$\{\langle t.name, t.phno \rangle \mid cust(t) \wedge t.city = "Delhi"\}$$

✓ Meaning: From `Cust`, retrieve `name` and `phno` of customers living in Delhi.

Aggregate Functions in Relational Algebra

These are special functions used for mathematical calculations on attributes of tuples.

Functions:

SUM → total of values

AVERAGE (AVG) → mean value

MAXIMUM (MAX) → largest value

MINIMUM (MIN) → smallest value

COUNT → number of tuples

Grouping in Relational Algebra

Sometimes we need to:

- Group tuples in a relation based on some attribute(s).
- Apply an aggregate function on each group.

This is similar to GROUP BY in SQL.

Notation

It is denoted by γ (script F):

$$\gamma_{\text{grouping attributes}} \text{ Functions List } (R)$$

- **grouping attributes** $\rightarrow$ attributes on which tuples are grouped.
- **Functions list** $\rightarrow$ set of aggregate functions applied.
- **R** $\rightarrow$ relation.

Example

Suppose relation:

```
Emp(empid, dept, salary)
```

👉 To find the **average salary in each department**:

$$\gamma_{\text{dept}} \text{ AVG}(\text{salary})(\text{Emp})$$

👉 To find the **count of employees in each department**:

$$\gamma_{\text{dept}} \text{ COUNT}(\text{empid})(\text{Emp})$$

Query 1

Retrieve employees by grouping on deptno and their average salary.

Relational Algebra:

$$\gamma_{Dno} \text{ Count}(Eno), \text{ Average}(sal) (EMP)$$

- Group by **Dno** (department number).
- Count number of employees (**Eno**).
- Compute average salary (**sal**).

✓ Equivalent SQL:

sql

```
SELECT Dno, COUNT(Eno) AS Count_Eno, AVG(Sal) AS Average_sal
FROM Emp
GROUP BY Dno;
```

Query 2

Count employees and find total amount of salary by grouping on deptno.

Relational Algebra:

$$\gamma_{Dno} \text{ Count}(Eno), \text{ Sum}(sal) (EMP)$$

- Group by Dno.
- Count employees.
- Find sum of salaries.

✓ Equivalent SQL:

sql

```
SELECT Dno, COUNT(Eno) AS Count_Eno, SUM(Sal) AS Sum_sal
FROM Emp
GROUP BY Dno;
```

Domain Relational Calculus (DRC)

A variable that ranges over the domain of attributes is called a domain variable.

The general form of a DRC expression is:

$$\{x_1, x_2, \dots, x_n \mid \text{Condition}(x_1, x_2, \dots, x_n)\}$$

Example 1

Relation:

Students(rollno, name, age, gender, deptname)

Query: Find rollno, name of students whose deptname is "eee".

Domain variables:

a = rollno, b = name, c = age, d = gender, e = deptname

Expression:

$$\{a, b \mid Students(a, b, c, d, e) \text{ AND } e = "eee"\}$$

Example 2

Query: Find name, deptname of all male students.

Expression:

$$\{b, e \mid Students(a, b, c, d, e) \text{ AND } d = "M"\}$$

Example 3

Relation:

Emp(eno, name, address, salary, deptno)

Query: Find name, address of all employees who either:

Work in Deptno = 4 and salary > 10000

OR

Work in Deptno = 5 and salary > 15000

Domain variables:

a = eno, b = name, c = address, d = salary, e = deptno

Expression:

$$\{b, c \mid Emp(a, b, c, d, e) \text{ AND } ((e = 4 \text{ AND } d > 10000) \text{ OR } (e = 5 \text{ AND } d > 15000))\}$$